Database Concepts, ER Modeling & Physical Storage

PBEOT504: Database Management System (S5)

Bachelor of Technology (B.Tech) — 2024 Scheme

Complete Technical Notes, Schema Architectures & ER Design Suite

Prepared from Course Materials, Lecture Slides & KTU Curriculum

Module 1 Complete Syllabus Coverage:

- Concepts & Overview of DBMS: File Processing vs DBMS, Data Independence
- Structured, Semi-Structured & Unstructured Data, Three-Schema Architecture
- Database Languages (DDL, DML, DCL, VDL), Interfaces & DBMS Classifications
- Entity-Relationship (ER) Model: Entities, Attributes, Relationships, Constraints
- Weak Entities, Identifying Relationships, (min, max) & Ternary Relationships
- Physical Data Organization: Blocking Factors, Heap Files & Indexing Calculations

Compiled August 17, 2026

Contents

1 Database System Concepts and Architecture 2
1.1 Introduction: File Processing Systems vs. Database Approach 2
1.2 Characteristics of the Database Approach 2
1.3 Classification of Data: Structured, Semi-Structured, and Unstructured 4
1.4 Data Models, Schemas, and Instances 4
1.5 The Three-Schema Architecture and Data Independence 4
1.6 Database Languages and User Classifications 5
2 The Entity-Relationship (ER) Conceptual Model 7
2.1 Basic Concepts: Entities, Attributes, and Keys 7
2.2 Relationships and Structural Constraints 7
2.3 Weak Entity Types and Identifying Relationships 9
2.4 Ternary Relationships (Degree 3) 9
2.5 Comprehensive ER Modeling Case Studies 11
3 Physical Data Organization and Indexing Structures 13
3.1 Physical Storage Terminology 13
3.2 Record Placement: Pinned vs. Unpinned (Spanned vs. Unspanned) 13
3.3 Heap Files (Unordered File Organization) 14
3.4 Database Indexing Fundamentals 14
3.5 Single-Level Ordered Indexes 14
3.6 Multi-Level Indexes and B/B+ Trees 15
4 Comprehensive Numerical Problem Suite 16
5 Quick Revision Sheet & Model Examination Questions 19
5.1 Master Summary Table of ER Notations and Physical Storage Formulas 19
5.2 Model Examination Questions (KTU S5 Pattern) 19

1 Database System Concepts and Architecture

Syllabus Requirement: Concept & Overview of Database Management Systems (DBMS) - Characteristics of Database system, Structured, semi-structured and unstructured data. Data Models and Schema - Three Schema architecture. Database Languages.

1.1 Introduction: File Processing Systems vs. Database Approach

Before the advent of Database Management Systems (DBMS), organizations stored data in traditional operating system flat files. In a **File-Processing System**, each application department maintained its own private data files along with custom programs to process them.

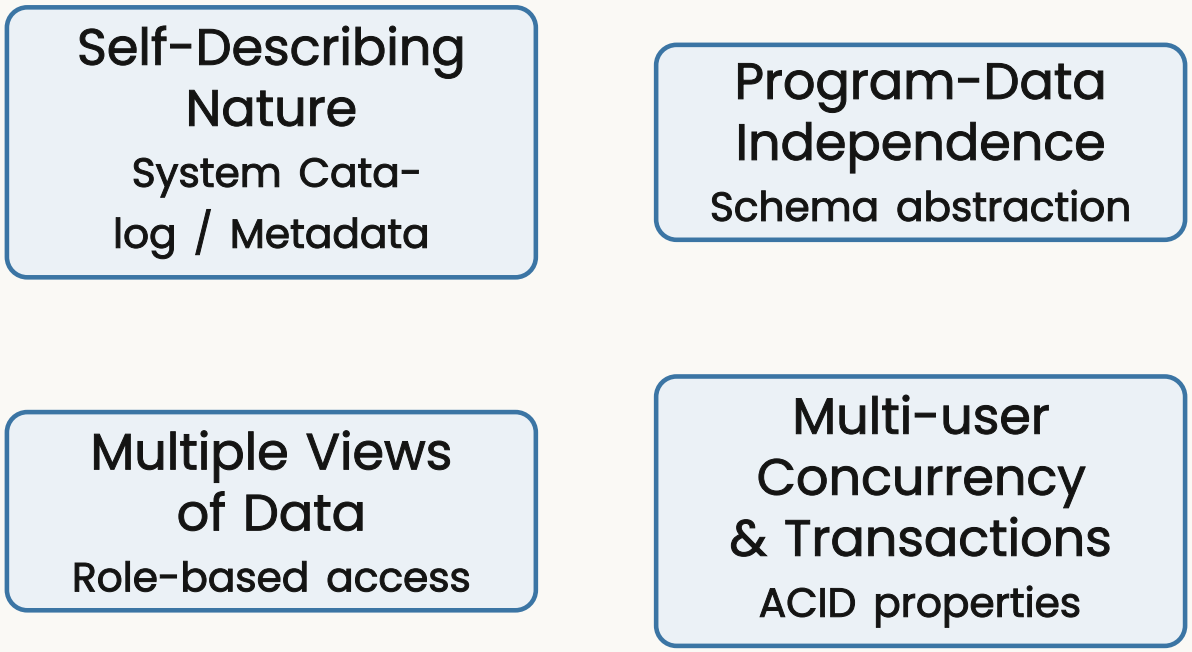
Limitations of Traditional File Processing

1. **Data Redundancy and Inconsistency:** The same data (e.g., student address) is duplicated across multiple department files. Updating one file without updating others leads to conflicting, inconsistent records.
2. **Program-Data Dependence:** The structure of data records is hardcoded directly into application programs. Modifying a record format requires rewriting and recompiling all dependent application programs.
3. **Difficulty in Accessing Data:** Ad-hoc queries (e.g., finding all students with GPA > 3.5 from a specific district) require writing entirely new application programs from scratch.
4. **Data Isolation and Lack of Standards:** Data is scattered in various file formats (binary, text, delimited), making data integration and cross-department queries exceptionally difficult.
5. **Integrity and Validation Problems:** Business rules (e.g., account balance ≥ 0) must be hard-coded inside every application program rather than enforced uniformly at the data level.
6. **Atomicity and Recovery Issues:** A system crash in the middle of a fund transfer may leave funds deducted from Account A without being credited to Account B.
7. **Concurrent Access Anomalies:** Uncontrolled simultaneous updates by multiple users lead to lost updates, dirty reads, and corrupted files.
8. **Security Problems:** Difficult to enforce granular user-level authorization and access control on raw operating system files.

Database and DBMS Defined: A **Database** is an organized, logically coherent collection of related data with some inherent meaning, designed to represent a mini-world (Universe of Discourse - UoD).
A **Database Management System (DBMS)** is a collection of generalized software programs that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications.

1.2 Characteristics of the Database Approach

The database approach possesses four fundamental characteristics that differentiate it from traditional file processing:



1. **Self-Describing Nature of a Database System:** A DBMS stores not only the operational data itself, but also a complete definition of the database structure, data types, constraints, and relationships. This descriptive data is called **Metadata** and is maintained in the **System Catalog (Data Dictionary)**.
2. **Insulation Between Programs and Data (Program-Data Independence):** The internal storage format and access paths can be modified without altering user application programs. The DBMS provides abstract interfaces so that programs refer to conceptual entities rather than physical byte offsets.
3. **Support for Multiple User Views:** A database has many distinct user groups (e.g., Students, Faculty, Accounts, Registrar). Each group requires a specialized, tailored perspective of the database. A view may present a virtual table derived from base tables while filtering out confidential fields.
4. **Sharing of Data and Multi-User Transaction Processing:** A multi-user DBMS enforces concurrency control mechanisms (locking, timestamps) ensuring that concurrent operations execute safely without interference, guaranteeing the **ACID Properties (Atomicity, Consistency, Isolation, Durability)**.

1.3 Classification of Data: Structured, Semi-Structured, and Unstructured

Data Category	Structural Organization	Data Formats	DBMS Technology
Structured Data	Strict pre-defined schema with rigid tables, rows, and strongly typed columns	Relational tables, SQL records, Fixed-width CSV	Relational DBMS (Oracle, PostgreSQL, MySQL)
Semi-Structured Data	Self-describing schema embedded within data; tags and key-value hierarchies	JSON, XML, BSON, YAML	Document & Key-Value NoSQL (MongoDB, Redis, CouchDB)
Unstructured Data	No pre-defined conceptual schema or tabular model; heavy raw content	Plain text, PDF, images, video, audio streams, sensor logs	Object Storage, Vector DBs, Graph DBs, Data Lakes

1.4 Data Models, Schemas, and Instances

Data Model Categories

A data model is a collection of concepts used to describe the structure of a database (data types, relationships, constraints, and basic operations).

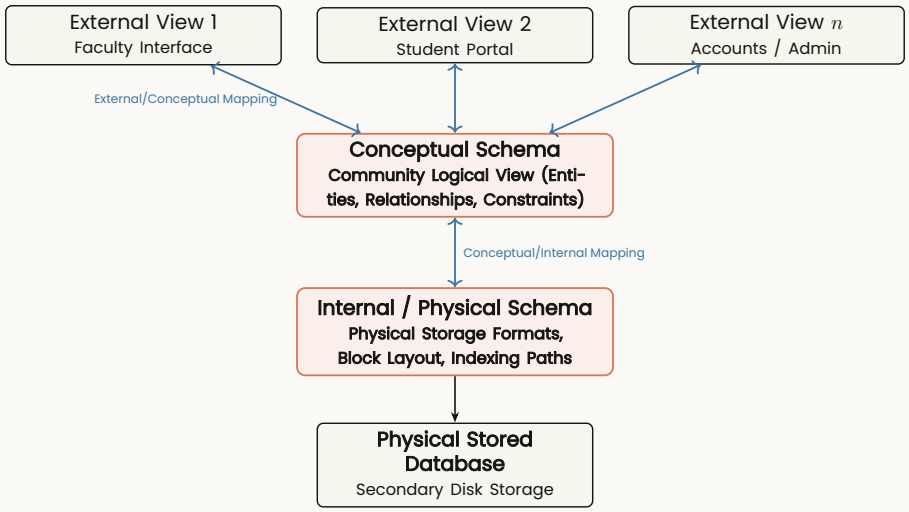
1. **Conceptual / High-Level Data Models:** Provide concepts close to how users perceive data (e.g., Entity-Relationship Model, Object Model).
2. **Representational / Implementation Data Models:** Concepts easily understood by end-users that map directly to physical computer storage (e.g., Relational Model, Network Model, Hierarchical Model).
3. **Physical / Low-Level Data Models:** Describe how data is stored on disk drives as records, blocks, orderings, and access paths (indexes).

Database Schema vs. Database State (Instance)

- **Database Schema (Intension):** The overall structural description of the database specified during design phase. It rarely changes. E.g., STUDENT(RollNo, Name, Age, Branch).
- **Database State / Instance (Extension):** The actual data stored in the database at a specific snapshot in time. It changes constantly as records are inserted, deleted, or updated.

1.5 The Three-Schema Architecture and Data Independence

The ANSI/SPARC **Three-Schema Architecture** separates user application interfaces from the physical database to achieve program-data independence.



The Three Schema Levels

1. **External Level (User Views):** Describes the part of the database relevant to a specific user group while concealing the rest of the database.
2. **Conceptual Level (Community Logical View):** Describes the structure of the entire database for a community of users, defining entities, data types, relationships, user operations, and semantic constraints without referencing physical storage details.
3. **Internal Level (Physical Storage Schema):** Describes the complete physical storage structure, record formats, page allocations, clustering, and access paths (indexes).

Data Independence

Data independence is the capacity to change the schema at one level of a database system without having to alter the schema at the next higher level.

Logical vs. Physical Data Independence: **1. Logical Data Independence:** The capacity to modify the **conceptual schema** without having to change external schemas or rewrite existing application programs.
Examples: Adding a new entity type or attribute, deleting an unused attribute, expanding relationship constraints.
Note: Challenging to achieve because application programs are heavily coupled to logical structures.

2. Physical Data Independence: The capacity to modify the **internal schema** without having to change the conceptual or external schemas.
Examples: Creating a new B+ tree index, changing physical file organization from heap to hashed, reorganizing disk blocks, switching storage hardware.
Note: Easy to achieve; only internal-to-conceptual mapping rules need adjustment.

1.6 Database Languages and User Classifications

Database Languages

- **DDL (Data Definition Language):** Used by DBAs and designers to define conceptual and internal schemas (e.g., `CREATE`, `ALTER`, `DROP`).
- **DML (Data Manipulation Language):** Used to retrieve, insert, delete, and modify data:
  - *High-Level/Non-procedural (Declarative) DML:* Specifies **what** data to retrieve rather than **how** to retrieve it (e.g., SQL). Processes sets of records simultaneously.

    - *Low-Level/Procedural DML:* Embedded in general-purpose programming languages; retrieves individual records one at a time and specifies exact navigation procedures.
- **VDL (View Definition Language):** Specifies user views and their mappings to the conceptual schema.
- **DCL (Data Control Language):** Controls user privileges and transaction states (e.g., `GRANT`, `REVOKE`, `COMMIT`, `ROLLBACK`).

Database Users

1. **Database Administrator (DBA):** Responsible for authorizing database access, monitoring performance, acquiring hardware/software resources, managing security, and overseeing backup/recovery.
2. **Database Designers:** Identify data requirements, choose appropriate data models, and create conceptual and physical schemas.
3. **End Users:** People whose jobs require access to the database (Casual users, Naive/Parametric users accessing predefined canned transactions, Sophisticated analysts).
4. **Application Programmers:** Software engineers who write DML queries embedded in host programming languages (Java, Python, C++) to build business applications.

2 The Entity-Relationship (ER) Conceptual Model

Syllabus Requirement: ER model - Basic concepts, entity set & attributes, notations, Relationships and constraints, cardinality, participation, notations, weak entities, relationships of degree 3.

2.1 Basic Concepts: Entities, Attributes, and Keys

The **Entity-Relationship (ER) Model** is a high-level conceptual data model introduced by Peter Chen in 1976. It represents the enterprise schema graphically as entities, attributes, and relationships.

1. Entities and Entity Sets

- **Entity:** A distinct "thing" or "object" in the real world that is distinguishable from other objects (e.g., a particular employee *John Smith*, a specific department *Research*).
- **Entity Type:** A collection of entities that share the same attributes (e.g., `EMPLOYEE`, `DEPARTMENT`).
- **Entity Set:** The current collection of all entities of a particular entity type stored in the database at any moment.

2. Types of Attributes

Attribute Type	Definition	Real-World Example	ER Diagram Notation
Simple (Atomic)	Cannot be divided into smaller components	`Age`, `Salary`, `Gender`	Single Oval
Composite	Can be divided into smaller sub-components	`Name` → (`First_Name`, `Last_Name`); `Address` → (`Street`, `City`)	Tree of connected Ovals
Single-Valued	Has exactly one single value for an entity	`SSN`, `DOB`, `RollNo`	Single Oval
Multi-Valued	Can have a set of multiple values for an entity	`Phone_Numbers`, `College_Degrees`	**Double Oval**
Derived	Value is dynamically calculated from stored attributes	`Age` (derived from `DOB`); `Total_Employees`	**Dashed Oval**
Key Attribute	Uniquely identifies each entity in the entity set	`Employee_ID`, `Course_Code`	Oval with **Underlined Text**

2.2 Relationships and Structural Constraints

1. Relationship Types, Sets, and Degree

- **Relationship:** An association among two or more entities (e.g., *John Smith* `WORKS_IN` *Research Department*).

- **Relationship Set:** A mathematical relation on entity sets $E_1, E_2, \ldots, E_n$:

$$R \subseteq \{(e_1, e_2, \ldots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \ldots, e_n \in E_n\} \tag{1}$$

- **Degree of a Relationship:** The number of participating entity types:
  1. **Unary / Recursive (Degree 1):** The same entity type participates more than once in different roles (e.g., an `EMPLOYEE` `SUPERVISES` another `EMPLOYEE` in roles of *Supervisor* and *Supervisee*).
  2. **Binary (Degree 2):** Two distinct entity types participate (e.g., `STUDENT` `ENROLLS` in `COURSE`).
  3. **Ternary (Degree 3):** Exactly three distinct entity types participate simultaneously (e.g., `SUPPLIER` `SUPPLIES` `PARTS` to `PROJECT`).

2. Cardinality Ratios for Binary Relationships

Specifies the maximum number of relationship instances that an entity can participate in:

1. **One-to-One ($1:1$):** An entity in A is associated with at most one entity in B, and vice versa (e.g., an `EMPLOYEE` `MANAGES` at most one `DEPARTMENT`).
2. **One-to-Many ($1:N$):** An entity in A can be associated with any number of entities in B, but an entity in B is associated with at most one entity in A (e.g., a `DEPARTMENT` `OFFERS` many `COURSES`, but a `COURSE` belongs to one `DEPARTMENT`).
3. **Many-to-One ($N:1$):** Multiple entities in A associate with one entity in B (e.g., many `EMPLOYEES` `WORK_FOR` one `DEPARTMENT`).
4. **Many-to-Many ($M:N$):** An entity in A can associate with many entities in B, and vice versa (e.g., `STUDENT` `ENROLLS` in multiple `COURSES`; a `COURSE` has multiple `STUDENTS`).

3. Participation Constraints (Existence Dependencies)

Specifies the minimum number of relationship instances that each entity must participate in:

1. **Total Participation (Existence Dependency):** Every entity in the entity set **must** participate in at least one relationship instance. Represented by a **Double Line**.
   Example: Every `LOAN` must be associated with a `CUSTOMER` (`LOAN` has total participation in `BORROWER`).
2. **Partial Participation:** Some entities in the entity set may **not** participate in the relationship. Represented by a **Single Line**.
   Example: Not every `CUSTOMER` takes a loan (`CUSTOMER` has partial participation).

4. The (min, max) Structural Constraint Notation

An alternative, precise constraint notation where a pair of integers (min, max) is placed on each edge connecting an entity type to a relationship:

$$0 \leq min \leq max \quad \text{and} \quad max \geq 1 \tag{2}$$

- If $min = 0$, participation is **Partial**.
- If $min \geq 1$, participation is **Total**.
- *Example:* An edge from `PROFESSOR` to `TEACHES` marked $(0, 4)$ indicates a professor teaches a minimum of 0 and a maximum of 4 courses. An edge from `COURSE` to `TEACHES` marked $(1, 3)$ indicates each course is taught by at least 1 and at most 3 professors.

2.3 Weak Entity Types and Identifying Relationships

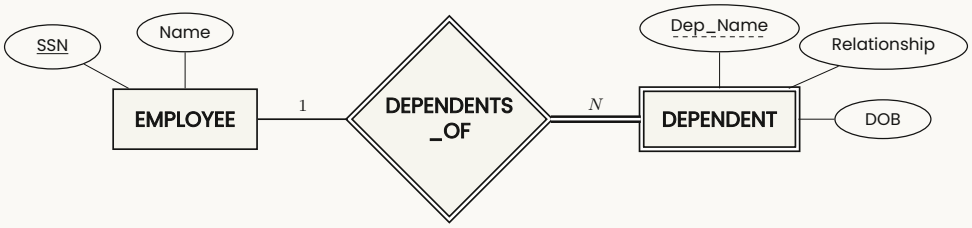
Weak Entity Type Defined: An entity type that **does not have a primary key attribute of its own** is called a **Weak Entity Type.**
Entities belonging to a weak entity type are identified by being associated with specific entities from another entity type called the **Owner / Identifying Entity Type** via an **Identifying Relationship.**

Key Characteristics of Weak Entity Types

1. **Partial Key (Discriminator):** Weak entities possess a partial key (set of attributes that can uniquely distinguish weak entities related to the *same* owner entity). Represented by a **Dashed Underline** (Attribute).
2. **Composite Primary Key Formulation:**

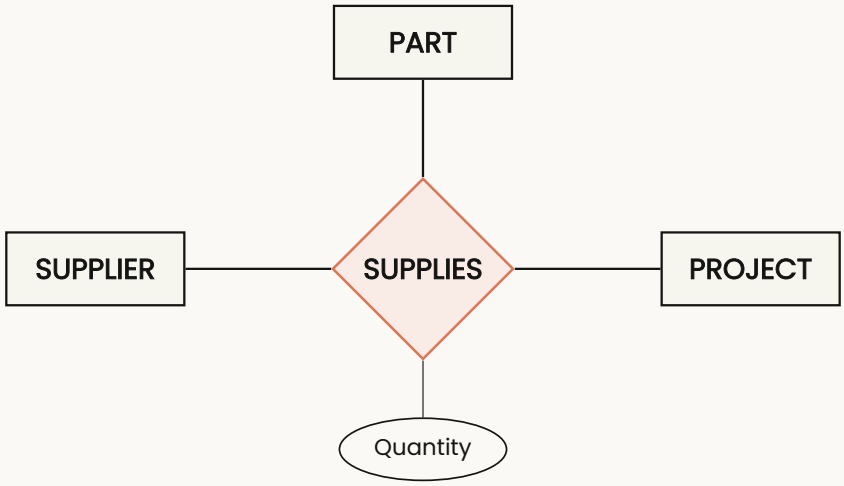
$$\text{Primary Key of Weak Entity} = \{\text{Primary Key of Owner Entity}\} \cup \{\text{Partial Key of Weak Entity}\} \tag{3}$$

3. **Mandatory Total Participation:** A weak entity cannot exist without its owner; hence, a weak entity **always has total participation (double line)** in its identifying relationship.
4. **Diagram Notations:**
   - Weak Entity Type: **Double Rectangle.**
   - Identifying Relationship: **Double Diamond.**



2.4 Ternary Relationships (Degree 3)

A **Ternary Relationship** represents an atomic association connecting three distinct entity types simultaneously.



Ternary Relationship vs. Three Binary Relationships

A single ternary relationship `SUPPLIES(Supplier, Part, Project)` cannot generally be decomposed into three binary relationships (`Supplies_Part`, `Part_UsedIn_Project`, `Supplier_For_Project`). The ternary relationship captures the exact fact that *Supplier* S_1 *supplied Part* P_1 *specifically for Project* J_1, whereas binary combinations only represent pairwise associations without tying all three components together.

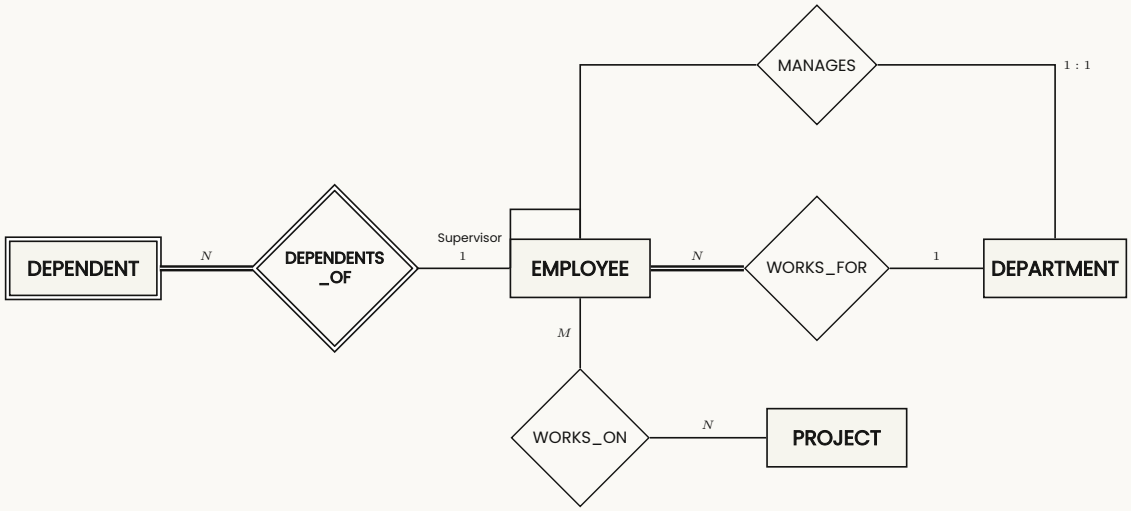
2.5 Comprehensive ER Modeling Case Studies

Case Study 1: Enterprise Company Database

Design an ER schema for a company with the following requirements:

- `DEPARTMENT` has unique name, number (`DNumber`), and multiple locations.
- `EMPLOYEE` has `SSN` , name, address, salary, DOB. An employee works in one department and can manage one department. An employee supervises other employees.
- `PROJECT` has name, number (`PNumber`), location, controlled by one department. Employees work on multiple projects with recorded hours.
- `DEPENDENT` has name, gender, DOB, relationship to employee (weak entity).

Step-by-Step Solution



Case Study 2: Television Reality Show Database (KTU Scenario Problem)

Design an ER schema for TV Reality Shows:

- `REALITY_SHOW` : Name, Genre, Basic_Info, Participant_Names (≥ 2 participants).
- `PRODUCER` : Company_Name, Company_Country. A show is produced by exactly 1 producer; 1 producer produces 1 show ($1:1$).
- `TELEVISION` : Name, Start_Year, Head_Office. Broadcasts multiple shows; each show broadcast by 1 TV ($1:N$).
- `USER` : Username, Password, Age. A user rates multiple shows, a show rated by multiple users ($M:N$) with score (0 to 10).

Step-by-Step Solution

1. **Entities & Primary Keys:** `REALITY_SHOW` (Show_ID, Name, Genre, Basic_Info), `PRODUCER` (Company_Name, Country), `TELEVISION` (TV_Name, Start_Year, Head_Office), `USER` (Username, Password, Age).
2. **Multivalued Attribute:** `Participants` (double oval connected to `REALITY_SHOW`).
3. **Relationships & Cardinalities:**
   - `PRODUCES` : Binary $1:1$ between `PRODUCER` and `REALITY_SHOW` .
   - `BROADCASTS` : Binary $1:N$ between `TELEVISION` and `REALITY_SHOW` .

- `RATES`: Binary $M : N$ between `USER` and `REALITY_SHOW` with relationship attribute `Rating_Score`.

3 Physical Data Organization and Indexing Structures

Syllabus Requirement: Physical Data Organization - Review of terms: physical and logical records, blocking factor, pinned and unpinned organization. Heap files, Indexing, Singe level indices, numerical examples, Multi-level-indices, numerical examples

3.1 Physical Storage Terminology

Databases are stored permanently on non-volatile secondary storage devices (magnetic hard disks, solid-state drives). Data transfer between disk and main memory (RAM) occurs in fixed-size units called **Disk Blocks (Pages)**.

Key Storage Parameters

- **Logical Record (R):** A single data entity tuple consisting of multiple fields (e.g., student record of size $R = 150$ bytes).
- **Disk Block Size (B):** The fixed unit of physical data transfer (typically $B = 512, 1024, 2048,$ or 4096 bytes).
- **Total Number of Records (r):** Number of records in the data file.

Blocking Factor (bfr)

The **Blocking Factor** (bfr) is the number of whole logical records that can be stored in a single disk block:

$$bfr = \left\lfloor \frac{B}{R} \right\rfloor \quad \text{[records per block]} \tag{4}$$

The total number of physical disk blocks (b) required to store the data file of r records is:

$$b = \left\lceil \frac{r}{bfr} \right\rceil \quad \text{[disk blocks]} \tag{5}$$

The unused (wasted) space in each disk block is:

$$\text{Unused Space per Block} = B - (bfr \times R) \quad \text{[bytes]} \tag{6}$$

3.2 Record Placement: Pinned vs. Unpinned (Spanned vs. Unspanned)

Organization	Physical Mechanism	Engineering Trade-off
Unspanned (Pinned)	Each record is contained entirely within a single block. If a record does not fit in remaining block bytes, the space is left unused (padded) and the record starts in the next block.	Simpler record retrieval; no multi-block pointer overhead; wastes space when record size R is large relative to block size B.
Spanned (Unpinned)	Records can cross block boundaries. A record starts in one block and continues into the next block using a pointer.	Zero wasted space; mandatory when record size $R > B$; requires two disk block I/Os to read a spanned record.

3.3 Heap Files (Unordered File Organization)

In a **Heap File**, records are placed on disk in the order they are inserted. New records are appended to the last block of the file.

- **Insertion Cost:** Fast ($O(1)$ block I/Os, simply write to the last block).
- **Search Cost (Linear Search):**

$$\text{Average Block Accesses} = \left\lfloor \frac{b}{2} \right\rfloor, \qquad \text{Worst-Case Block Accesses} = b \tag{7}$$

- **Deletion Cost:** Requires finding the record, setting a deletion flag or moving the last record into the deleted slot.

3.4 Database Indexing Fundamentals

An **Index** is an auxiliary access structure built on a search-key field that accelerates data retrieval, reducing search time from linear $O(b)$ to logarithmic $O(\log_2 b)$ or constant $O(1)$.

An index file consists of ordered index entries:

$$\text{Index Entry} = \langle K_i, P_i \rangle \tag{8}$$

where K_i is the search key value and P_i is a block pointer (address of the disk block containing the record).

Let index record size be $R_i = V_K + P_B$ (where V_K is key size and P_B is pointer size). The **Index Blocking Factor** is:

$$bfr_i = \left\lfloor \frac{B}{R_i} \right\rfloor \quad \text{[index entries per block]} \tag{9}$$

3.5 Single-Level Ordered Indexes

Index Type	Data File Ordering	Search Key Field	Index Density & Size
Primary Index	Ordered by search key	Primary Key (Unique)	**Sparse Index:** One entry per data block (anchor record). Number of entries $= b$.
Clustering Index	Ordered by search key	Non-Key (Duplicate values)	**Sparse Index:** One entry per distinct search value.
Secondary Index	Unordered (or ordered on a different field)	Candidate Key / Non-Key	**Dense Index:** One entry per data record (r entries) for key fields.

Search Cost Comparison: Unordered Heap vs. Primary Index

Consider a data file with $r = 100,000$ records, record size $R = 200$ bytes, block size $B = 2048$ bytes.
Key size $V_K = 10$ bytes, block pointer $P_B = 6$ bytes.
1. **Data File Parameters:**

$$bfr = \left\lfloor \frac{2048}{200} \right\rfloor = 10 \text{ records/block}, \qquad b = \left\lceil \frac{100,000}{10} \right\rceil = 10,000 \text{ blocks}$$

2. **Linear Search on Heap File:**

$$\text{Average Cost} = \frac{b}{2} = \frac{10,000}{2} = \mathbf{5,000} \text{ block accesses}$$

3. **Primary Index Parameters (Sparse):**

$$R_i = 10 + 6 = 16 \text{ bytes}, \qquad bfr_i = \left\lfloor \frac{2048}{16} \right\rfloor = 128 \text{ entries/block}$$

Number of primary index entries $= b = 10,000$. Total index blocks (b_i):

$$b_i = \left\lceil \frac{10,000}{128} \right\rceil = 79 \text{ index blocks}$$

4. **Binary Search using Primary Index:**

$$\text{Cost} = \lceil \log_2(b_i) \rceil + 1 = \lceil \log_2(79) \rceil + 1 = 7 + 1 = \mathbf{8} \text{ block accesses}$$

Result: Indexing reduces disk I/O from **5,000 block accesses down to just 8 block accesses (a 625× speedup!)**.

3.6 Multi-Level Indexes and B/B+ Trees

When the primary index file itself becomes too large to fit in main memory, a **Second-Level Index** is created on top of the primary index, treating the primary index as a data file. This process is repeated hierarchically until the top-level index fits in a single block (the root block).

Multi-Level Index Fan-Out and Tree Height: The **Fan-Out** (fo) of an index block is the number of pointers it can store:

$$fo = bfr_i = \left\lfloor \frac{B}{V_K + P_B} \right\rfloor \tag{10}$$

For a base index of b_1 blocks, the number of index levels (x) is:

$$x = \left\lceil \log_{fo}(b_1) \right\rceil + 1 \tag{11}$$

Total Block Accesses to retrieve a record $= x + 1$ (where $+1$ is for the final data block).

4 Comprehensive Numerical Problem Suite

Problem 1: Blocking Factor and File Storage Size

A customer data file contains $r = 50,000$ fixed-length records. Each record has size $R = 250$ bytes. The disk block size is $B = 4096$ bytes. Calculate: (a) Blocking factor (bfr), (b) Total number of disk blocks (b) using unspanned organization, (c) Total wasted storage space across the file, and (d) Average block accesses to search a record in heap organization.

Step-by-Step Solution

1. Blocking Factor:

$$bfr = \left\lfloor \frac{B}{R} \right\rfloor = \left\lfloor \frac{4096}{250} \right\rfloor = \lfloor 16.384 \rfloor = \mathbf{16} \text{ records/block}$$

2. Total Data Blocks:

$$b = \left\lceil \frac{r}{bfr} \right\rceil = \left\lceil \frac{50,000}{16} \right\rceil = \lceil 3125 \rceil = \mathbf{3,125} \text{ blocks}$$

3. Wasted Space:

$$\text{Unused bytes per block} = 4096 - (16 \times 250) = 4096 - 4000 = 96 \text{ bytes}$$
$$\text{Total Wasted Space} = 3,125 \times 96 \text{ bytes} = \mathbf{300,000} \text{ bytes} \approx \mathbf{300} \text{ kB}$$

4. Average Linear Search Accesses (Heap):

$$\text{Cost} = \left\lfloor \frac{b}{2} \right\rfloor = \left\lfloor \frac{3125}{2} \right\rfloor = \mathbf{1,562} \text{ block accesses}$$

Problem 2: Primary Index Storage and Search Performance

For the file in Problem 1, suppose the records are ordered on a primary key field of size $V_K = 14$ bytes. Block pointers take $P_B = 6$ bytes. Construct a primary index and calculate: (a) Number of index entries, (b) Index blocking factor, (c) Number of index blocks (b_i), and (d) Total block accesses to find a record using binary search on the index.

Step-by-Step Solution

1. Primary Index is **sparse** (one entry per data block).

$$\text{Number of Index Entries} = b = \mathbf{3,125} \text{ entries}$$

2. Index record size: $R_i = V_K + P_B = 14 + 6 = 20$ bytes.
3. Index Blocking Factor:

$$bfr_i = \left\lfloor \frac{B}{R_i} \right\rfloor = \left\lfloor \frac{4096}{20} \right\rfloor = \lfloor 204.8 \rfloor = \mathbf{204} \text{ entries/block}$$

4. Number of Index Blocks:

$$b_i = \left\lceil \frac{3,125}{204} \right\rceil = \lceil 15.318 \rceil = \mathbf{16} \text{ index blocks}$$

5. Binary Search Block Accesses:

$$\text{Cost} = \lceil \log_2(b_i) \rceil + 1 = \lceil \log_2(16) \rceil + 1 = 4 + 1 = \textbf{5} \text{ block accesses}$$

Problem 3: Secondary Index on a Non-Ordering Key Field

Suppose an unordered employee file of $r = 100,000$ records has record size $R = 100$ bytes and block size $B = 2048$ bytes. A secondary index is built on a non-ordering unique SSN field of size $V_K = 12$ bytes and record pointer $P_R = 8$ bytes. Calculate: (a) Total data blocks, (b) Total secondary index blocks, and (c) Search cost comparison between heap linear search and secondary index.

Step-by-Step Solution

1. Data File Parameters:

$$bfr = \left\lfloor \frac{2048}{100} \right\rfloor = 20 \text{ records/block}, \qquad b = \left\lceil \frac{100,000}{20} \right\rceil = \textbf{5,000} \text{ data blocks}$$

2. Secondary Index is **dense** (one entry per data record):

$$\text{Number of Index Entries} = r = 100,000$$

Index entry size: $R_i = V_K + P_R = 12 + 8 = 20$ bytes.

$$bfr_i = \left\lfloor \frac{2048}{20} \right\rfloor = 102 \text{ entries/block}$$

Total secondary index blocks:

$$b_i = \left\lceil \frac{100,000}{102} \right\rceil = \lceil 980.39 \rceil = \textbf{981} \text{ index blocks}$$

3. Search Cost Comparison:
   - **Unordered Heap Search:** Average $\lfloor b/2 \rfloor = \textbf{2,500}$ block accesses.
   - **Secondary Index Search:** $\lceil \log_2(b_i) \rceil + 1 = \lceil \log_2(981) \rceil + 1 = 10 + 1 = \textbf{11}$ block accesses.

Problem 4: Multi-Level Index Access Cost

A large data file has $b = 1,000,000$ data blocks. A multi-level index is constructed with block size $B = 4096$ bytes, search key size $V_K = 26$ bytes, and block pointer $P_B = 6$ bytes. Determine: (a) Fan-out of the index, (b) Number of index blocks at each level, (c) Number of levels in the index tree, and (d) Total disk block accesses to retrieve any record.

Step-by-Step Solution

1. Fan-Out (fo):

$$R_i = 26 + 6 = 32 \text{ bytes} \implies fo = bfr_i = \left\lfloor \frac{4096}{32} \right\rfloor = \textbf{128} \text{ entries/block}$$

2. Multi-Level Index Hierarchy:

- **Level 1 Index Blocks (b_1):** $\lceil \frac{b}{fo} \rceil = \lceil \frac{1{,}000{,}000}{128} \rceil = \mathbf{7,813}$ blocks.
- **Level 2 Index Blocks (b_2):** $\lceil \frac{b_1}{fo} \rceil = \lceil \frac{7{,}813}{128} \rceil = \mathbf{62}$ blocks.
- **Level 3 Index Blocks (b_3):** $\lceil \frac{b_2}{fo} \rceil = \lceil \frac{62}{128} \rceil = \mathbf{1}$ block (Root level reached!).

3. Total Index Levels: $x = \mathbf{3}$ levels.
4. Total Disk Accesses to retrieve record $= x + 1 = 3 + 1 = \mathbf{4}$ block accesses (Level 3 root $\rightarrow$ Level 2 $\rightarrow$ Level 1 $\rightarrow$ Data Block).

5 Quick Revision Sheet & Model Examination Questions

5.1 Master Summary Table of ER Notations and Physical Storage Formulas

Concept	Notation / Formula	Meaning & Description
Strong Entity	Rectangle	Entity with a primary key attribute
Weak Entity	Double Rectangle	Dependent entity without primary key of its own
Partial Key	Dashed Underline (Attr)	Discriminator within identifying entity set
Relationship	Diamond	Association among entities
Identifying Rel	Double Diamond	Relates weak entity to owner entity
Multivalued Attr	Double Oval	Attribute with set of multiple values
Derived Attribute	Dashed Oval	Dynamically computed attribute
Total Participation	Double Line	Existence dependency ($min \geq 1$)
Blocking Factor	$bfr = \lfloor B/R \rfloor$	Records per physical disk block
Primary Index	$b_i = \lceil b/bfr_i \rceil$	Sparse index on ordered primary key
Secondary Index	$b_i = \lceil r/bfr_i \rceil$	Dense index on candidate key
Multi-Level Cost	Cost $= x + 1$ block I/Os	$x = \lceil \log_{fo}(b_1) \rceil + 1$ levels in index tree

5.2 Model Examination Questions (KTU S5 Pattern)

University Examination Pattern (PBEOT504 Scheme)

The End Semester Examination consists of **Part A** (8 compulsory questions of 2 marks each, 2 from each module) and **Part B** (Module choice questions carrying 6 marks each).

Part A: Short Answer Questions (2 Marks Each)

1. **Q1:** State two major limitations of traditional file processing systems that are overcome by a DBMS.
2. **Q2:** Differentiate between *Logical Data Independence* and *Physical Data Independence*. Which one is more difficult to achieve and why?
3. **Q3:** Define a *Weak Entity Type*. How is it represented in an ER diagram?
4. **Q4:** What is meant by *Blocking Factor* (bfr)? How does it affect total disk blocks required for a data file?
5. **Q5:** Differentiate between a *Primary Index* and a *Secondary Index* in terms of file ordering and index density.
6. **Q6:** Explain the (min, max) notation used to represent structural constraints in ER diagrams.

Part B: Long Answer / Analytical Questions (6 Marks Each)

1. **Q7 (a):** Explain the ANSI/SPARC Three-Schema Architecture with a neat diagram. Describe the role of mappings between adjacent schema levels. (4 Marks)

Q7 (b): Classify structured, semi-structured, and unstructured data with suitable examples. (2 Marks)

2. **Q8 (a):** A company database records information about projects, employees, and departments. An employee has an ID, name, and address. A department has a department number and name. A project has a project number and location. An employee works in one department, manages at most one department, and works on multiple projects with recorded hours. Draw the complete ER diagram showing keys, cardinality ratios, and participation constraints. (6 Marks)
3. **Q9 (a):** A data file contains $40,000$ records of size 200 bytes each. The block size is 2048 bytes. A primary index is built on a key field of 14 bytes with block pointer size of 6 bytes. Calculate: (i) Blocking factor of data file and index file, (ii) Total data blocks and index blocks, and (iii) Number of block accesses to search a record using binary search on the index compared to linear search on the heap file. (6 Marks)

— End of Module 1 Study Notes —